

Histórias de usuários:

- US01: Como usuário, quero criar uma conta com e-mail e senha, para acessar o sistema financeiro.
- US02: Como usuário, quero fazer login na plataforma, para acessar meus dados financeiros.
- US03: Como usuário, quero cadastrar minha renda mensal, para controlar meu orçamento.
- US04: Como usuário, quero visualizar minha renda total do mês, para saber quanto posso gastar.
- US05: Como usuário, quero cadastrar meus gastos, para controlar minhas despesas.
- US06: Como usuário, quero classificar um gasto como essencial ou não essencial, para organizar minhas prioridades financeiras.
- US07: Como usuário, quero visualizar um resumo financeiro mensal com total de renda e total de gastos, para analisar meu saldo.
- US08: Como usuário, quero visualizar gráficos de gastos por categoria, para facilitar a análise visual por despesas.

Backlogs:

- | | |
|--|-----|
| 1. Desenvolver design do aplicativo | (3) |
| 2. Implementar cadastro e login de usuário | (5) |
| 3. Implementar cadastro de renda e gastos do usuário | (5) |
| 4. Implementar categorização de gastos | (3) |
| 5. Desenvolver registros e gráficos financeiros | (5) |
| 6. Integrar sistema com API | (8) |
| 7. Realizar testes e ajustes | (3) |

Análise INVEST:

US01:

I – Não depende de outras histórias.

N – Regras como validação de senha e confirmação por e-mail podem ser ajustadas.

V – Permite acesso ao sistema.

E – Uma Sprint.

S – Pode crescer se incluir verificação de e-mail e criptografia avançada.

T – Testes de cadastro válido/inválido são possíveis.

US02:

I – Não depende de outras histórias.

N – Pode negociar autenticação simples ou multifator.

V – Permite acesso aos dados financeiros do usuário.

E – Uma Sprint.

S – Escopo pequeno e bem definido.
T – Testes com credenciais válidas e inválidas são possíveis.

US03:

I – Não depende de outras histórias.
N – Pode negociar tipos de renda (fixa ou variável).
V – Permite o controle do orçamento mensal.
E – Uma Sprint.
S – Funcionalidade simples e objetiva.
T – Testes de cadastro e validação de valores são possíveis.

US04:

I – Não depende de outras histórias.
N – Pode negociar filtros por período.
V – Ajuda o usuário a entender quanto pode gastar.
E – Uma Sprint.
S – Escopo pequeno.
T – Testes de cálculo e exibição do valor total são possíveis.

US05:

I – Não depende de outras histórias.
N – Campos e categorias de gastos podem ser ajustados.
V – Permite o controle das despesas do usuário.
E – Uma Sprint.
S – Pode crescer se incluir recorrência ou anexos.
T – Testes de cadastro e persistência de gastos são possíveis.

US06:

I – Não depende de outras histórias.
N – Tipos de classificação podem ser alterados.
V – Ajuda o usuário a organizar prioridades financeiras.
E – Uma Sprint.
S – Escopo pequeno e bem definido.
T – Testes de classificação e alteração de tipo são possíveis.

US07:

- I – Não depende de outras histórias.
- N – Layout e métricas do resumo podem ser ajustados.
- V – Permite analisar o saldo mensal.
- E – Uma Sprint.
- S – Escopo pequeno e objetivo.
- T – Testes de cálculo de saldo são possíveis.

US08:

- I – Não depende de outras histórias.
- N – Tipos de gráficos e categorias podem ser negociados.
- V – Facilita a análise visual das despesas.
- E – Uma Sprint.
- S – Pode crescer se incluir filtros avançados.
- T – Testes de geração de gráficos com dados reais são possíveis.

MoSCow:

MUST HAVE:

US01: Sem uma conta, o usuário não consegue acessar a aplicação.

US02: Sem o login, o usuário não terá um acesso personalizado aos seus dados financeiros.

US03: Sem o cadastro de renda, não é possível analisar o controle financeiro do usuário.

US05: Sem o cadastro de gastos, não é possível analisar o controle financeiro do usuário.

US07: Sem o resumo financeiro mensal, não será possível analisar o saldo do usuário.

SHOULD HAVE:

US04: A renda total do mês estará no resumo financeiro mensal

US06: Não é necessário classificar os gastos, porém ajuda na organização

COULD HAVE:

US08: Os gráficos são uma ferramenta visual para analisar os dados, porém é possível fazer isso pelos resumos financeiros.

WON'T HAVE

Critérios de Aceite:

US01:

FUNCIONAL 1:

Dado que o usuário está na tela de cadastro

Quando ele informa um e-mail válido e uma senha válida

Então o sistema deve criar a conta e permitir o acesso à plataforma

FUNCIONAL 2:

Dado que o e-mail informado já está cadastrado

Quando o usuário tenta criar uma nova conta

Então o sistema deve exibir uma mensagem de erro informando que o e-mail já está em uso

NÃO FUNCIONAL:

Dado que o usuário cria uma conta

Quando o sistema armazena a senha

Então a senha deve ser salva de forma criptografada

US02:

FUNCIONAL 1:

Dado que o usuário possui uma conta cadastrada

Quando ele informa e-mail e senha corretos

Então o sistema deve permitir o acesso aos dados financeiros

FUNCIONAL 2:

Dado que o usuário informa uma senha inválida

Quando tenta realizar o login

Então o sistema deve negar o acesso e exibir uma mensagem de erro

NÃO FUNCIONAL:

Dado que o usuário tenta realizar login

Quando envia suas credenciais

Então o sistema deve autenticar o usuário em até 2 segundos

US03:

FUNCIONAL 1:

Dado que o usuário está autenticado

Quando ele informa um valor positivo de renda mensal
Então o sistema deve salvar a renda com sucesso

FUNCIONAL 2:

Dado que o usuário informa um valor negativo ou vazio
Quando tenta salvar a renda
Então o sistema deve exibir uma mensagem de erro

NÃO FUNCIONAL:

Dado que a renda foi cadastrada
Quando o usuário realiza logout e login novamente
Então o sistema deve manter a renda salva

US04:

FUNCIONAL 1:

Dado que o usuário possui renda cadastrada
Quando acessa a tela de resumo financeiro
Então o sistema deve exibir o total da renda do mês atual

FUNCIONAL 2:

Dado que não há renda cadastrada para o mês
Quando o usuário acessa a tela de resumo
Então o sistema deve exibir o valor zero

NÃO FUNCIONAL:

Dado que o usuário está na tela de resumo financeiro
Quando ocorre atualização dos dados
Então o sistema deve atualizar as informações sem recarregar a página

US05:

FUNCIONAL 1:

Dado que o usuário está autenticado
Quando ele informa valor, descrição e data do gasto
Então o sistema deve salvar o gasto corretamente

FUNCIONAL 2:

Dado que o usuário informa um valor negativo
Quando tenta salvar o gasto
Então o sistema deve exibir uma mensagem de erro

NÃO FUNCIONAL:

Dado que o usuário cadastra um gasto

Quando confirma o envio
Então o sistema deve salvar o gasto em até 2 segundos

US06:

FUNCIONAL 1:

Dado que o gasto está cadastrado
Quando o usuário marca o gasto como essencial
Então o sistema deve atualizar a classificação do gasto

FUNCIONAL 2:

Dado que um gasto já possui classificação
Quando o usuário altera para não essencial
Então o sistema deve salvar a nova classificação

NÃO FUNCIONAL:

Dado que o usuário está na tela de gastos
Quando deseja classificar um gasto
Então o sistema deve permitir a classificação com no máximo dois cliques

US07:

FUNCIONAL 1:

Dado que o usuário possui renda e gastos cadastrados
Quando acessa o resumo mensal
Então o sistema deve exibir total de renda, total de gastos e saldo

FUNCIONAL 2:

Dado que o usuário possui apenas renda cadastrada
Quando acessa o resumo mensal
Então o sistema deve exibir o total de gastos como zero

NÃO FUNCIONAL:

Dado que o usuário acessa o resumo mensal
Quando as informações são exibidas
Então o sistema deve apresentar os dados de forma clara e legível

US08:

FUNCIONAL 1:

Dado que o usuário possui gastos cadastrados

Quando acessa a tela de gráficos

Então o sistema deve exibir um gráfico com os gastos por categoria

FUNCIONAL 2:

Dado que não há gastos registrados

Quando o usuário acessa os gráficos

Então o sistema deve exibir um gráfico vazio ou mensagem informativa

NÃO FUNCIONAL:

Dado que o usuário acessa a tela de gráficos

Quando os dados são carregados

Então os gráficos devem ser exibidos em até 3 segundos

Requisitos Não Funcionais:

1. Desenvolvimento:

- RNF1: O sistema deve ser desenvolvido utilizando a arquitetura cliente-servidor. (US01,US02,US03,US04,US05,US07)
- RNF2: O sistema deve ser implementado em Python com a FastAPI no back-end e implementado em JavaScript com React em todo front-end.(US08)
-

2. Segurança da informação:

- RNF3: O sistema deve validar todos os dados recebidos do usuários.(US01,US02,US03,US05)
- RNF4: O sistema deve armazenar as senhas de forma criptografada.(US01,US02)
- RNF5: O sistema deve impedir o acesso direto ao banco de dados através da API criando controle de acesso. (US03,US04,US05,US07)

3. Desempenho:

- RNF6: O sistema deve responder às requisições em até 5 segundos.(US02,US03,US05)
- RNF7: O sistema deve ser escalável para suportar um aumento progressivo de usuários sem perda de performance(US04,US05,US07)
- RNF8: O sistema deve consultar o banco de dados com queries otimizadas.(US07,US08)

4. Disponibilidade:

- RNF9: O sistema deve permanecer disponível durante o período de utilização.(US01,US03,US03,US05)
- RNF10: O sistema deve permanecer os dados armazenados após a reinicialização do servidor.(US03,US05,US07)
- RNF11: O sistema deve ter uma estratégia de recuperação que inclua backups regulares e processos automáticos de recuperação de falhas(US07)

5. Legislativo:

- RNF12: O sistema deve garantir a proteção e privacidade dos dados dos clientes segundo a lei nº 13.709/2018 (Brasil).(US01,US02)